



NAME : HAMDAN ALI

PROJECT: TASK INDEX

WEEK 5 : Day 25

DAY 25

PROJECT : Screenshots Of Output

CODEBASE Of the Work

```
src > components > ConfirmDialog.jsx > ...
1 import { useTaskActions, useTaskState } from '../context/TaskContext';
2 import './ConfirmDialog.css';
3
4 export default function ConfirmDialog() {
5   const { tasks, ui } = useTaskState();
6   const actions = useTaskActions();
7
8   if (!ui.pendingDeleteId) return null;
9
10  const task = tasks.find((t) => t.id === ui.pendingDeleteId);
11  if (!task) return null;
12
```

```
PS D:\tasks\task-manager-fixed\task-manager-fixed> npm run dev
use `oxc` instead.
`optimizeDeps.rollupOptions` / `ssr.optimizeDeps.rollupOptions` is deprecated. Use `optimizeDeps.rolldownOptions` instead
. Note that this option may be set by a plugin. Set VITE_DEPRECATION_TRACE=1 to see where it is called.
[vite:react-babel] We recommend switching to `@vitejs/plugin-react-oxc` for improved performance. More information at https://vite.dev/rolldown
Warning: Invalid input options (1 issue found)
- For the "jsx". Invalid key: Expected never but received "jsx".

VITE v8.2.1 ready in 543 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
Warning: Invalid input options (1 issue found)
- For the "jsx". Invalid key: Expected never but received "jsx".
```



Dark theme:

Dark theme:

EDIT OPTION:

There is an edit option here also , here we can multiple kind of changes like title , catregory ,priority etc

And Also D we can change the date inside Edit option:

This part shows Details about TaskIndex and when in each of these it will show the result, for example if click **personal then:**


TaskIndex
your working catalog

DRAWERS


All Tasks
5


Today
1


Upcoming
1


Completed
3

CATEGORIES


Work
1


Personal
1


Learning
0


Errands
0

RESULT


TaskIndex
your working catalog

DRAWERS


All Tasks
5


Today
1


Upcoming
1


Completed
3

CATEGORIES


Work
1


Personal
1


Learning
0


Errands
0

Personal

Search the catalog...


68%

OPEN
2

DUE TODAY
1

OVERDUE
0

☐

Call the internet provider about the outage
Low

Reference ticket #4471.

 Aug 18
Personal



There Is also An Option Of Search:

All Tasks

68%

OPEN
2

DUE TODAY
1

OVERDUE
0

Draft Q3 sprint retrospective notes

Pull velocity chart from Linear before the meeting.

Today

Work

High

We can also add New Task:

+ New Task

×

Sort

Priority

New task card

Title

e.g. Review the onboarding doc

Notes (optional)

Any detail future-you will want

Category

Work

Priority

Medium

Due date (optional)

mm/dd/yyyy

Cancel

Add to catalog



EXPLANATION OF THE PROJECT

DAY 25

TaskIndex

A React Task Manager, Built Like a Card Catalog

Full-Stack Web Development Internship — Week 5, Day 25

Project Focus: React Fundamentals

Hamdan Ali

What I Built

For Day 25 the brief was a frontend-only React task manager — no backend, just the fundamentals: components, props, state, hooks, forms, and lists. I did not want to build another plain checklist with a text box and an "Add" button, so before writing any code I spent time deciding what the app should actually feel like to use.

I settled on a library card catalog as the working idea. A task list is, functionally, an index of things you need to find again later — a due date instead of a shelf number, a category instead of a subject heading. That gave me a concrete visual language to build from instead of guessing at colors and spacing as I went: index cards for tasks, drawers for the different views in the sidebar, and a rubber-stamp badge for anything that is overdue.

The result is called TaskIndex. It runs entirely in the browser, keeps your tasks in localStorage so nothing is lost on refresh, and covers everything the brief asked for — plus a few things I added because they made the app feel finished rather than just functional: dark mode, a progress ring, a delete confirmation step, and two different empty states depending on whether the catalog is genuinely empty or just filtered down to nothing.

React Concepts This Project Covers

This was the assigned learning target for the week, so I made a point of touching each concept somewhere real in the app rather than in an isolated example:

- Function components and props — every piece of UI, from a single task card to the whole sidebar, is a component that receives what it needs through props.
- State with `useState` — theme, the mobile menu, and the add/edit form fields all live as local component state.
- `useReducer` for anything with more than two or three related state transitions — the UI reducer in `TaskContext` handles ten different action types (view changes, opening the form, requesting a delete, and so on).
- Context, to avoid passing the same task list and action functions down through five layers of components that do not otherwise need them.
- `useEffect` — for syncing the theme to the `<html>` element and `localStorage`, for autofocusing the form's title field, and for listening to the Escape key while a dialog is open.
- Conditional rendering — the empty states, the overdue stamp, the delete confirmation, and the edit-vs-create form all branch on this pattern.
- Lists and keys — `TaskList` maps the filtered array to `TaskCard` components, each keyed by its id.
- Controlled forms — every input in `TaskForm` is bound to state, with a basic validation check before it lets you submit an empty title.
- A custom hook (`useLocalStorage`) — this is the one piece that goes slightly past "fundamentals," but it is short, and it meant I did not have to repeat the same try/catch `localStorage` logic in two places.

How the Code Is Organized

I split the project into five folders under `src/`, each with one job:

`components/` — one component per file, each with its own CSS file next to it. I went back and forth on whether to use CSS Modules or a single global stylesheet, and landed on plain, scoped-by-convention CSS files (component-name.css imported directly into component-name.jsx) because it keeps related code physically next to each other without adding a build step.

`context/` — `TaskContext.jsx`, which is the only file that knows how tasks are actually stored and mutated.

hooks/ — useLocalStorage.js, a generic hook that works like useState but writes to localStorage on every change.

utils/ — taskUtils.js, which holds every pure function: date formatting, filtering, sorting, the stats calculation, and the seed data the app ships with.

styles/ — tokens.css (every color, font, spacing, and shadow value as a CSS variable) and global.css (resets and base typography).

The token file was probably the single most useful decision in the whole project. Because every color in every component references a variable like var(--color-primary) instead of a hex code, building the dark theme later was a matter of overriding about twenty variables under a [data-theme="dark"] selector — I did not have to touch a single component file to do it.

Why Context + useReducer Instead of Redux

The app is small enough that pulling in Redux would have been overkill — more boilerplate than the problem needed. But plain useState scattered across components would have meant passing tasks and update functions through props on every level, which gets messy fast once you have a sidebar, a header, a list, and two modals all needing to read or change the same data.

Context plus useReducer sits in the middle: I get the predictable, action-based updates that make a reducer nice to reason about, without adding a dependency. I did split the state into two providers internally (task data through one context, action functions through another) so that a component only reading tasks does not re-render just because an unrelated dispatch happened — a small thing, but it is the kind of detail that matters once the task list gets long.

Why localStorage Instead of a Backend

The brief was explicit that this is a frontend-only project, so there is no API and no database. localStorage was the obvious choice for making the app feel real rather than like a demo that resets itself every time you refresh the tab. The useLocalStorage hook keeps that logic in one place and wrapped in a try/catch, since private browsing mode and storage quota errors can make localStorage throw — I would rather the app quietly fall back to in-memory state than crash because a browser setting blocked writes.

Feature Walkthrough

Adding and Editing Tasks

One modal handles both. TaskForm checks whether `ui.editingTaskId` is set — if it is, the form pre-fills from the matching task and the submit button calls `updateTask` instead of `addTask`. I like this better than building two nearly identical forms, since any change to the fields (adding a new one, tweaking validation) only has to happen once.

Validation right now is intentionally minimal: a task needs a title, full stop. I thought about requiring a due date too, but tasks without a fixed deadline are a normal thing to want to track, so I left it optional and just changed how those tasks sort ("no due date" tasks fall to the bottom instead of floating to the top as if they were most urgent).

Filtering, Search, and Sort

The sidebar drawers (All, Today, Upcoming, Completed) and the category list both write to the same UI state, and selecting one resets the other — you are always looking at one clear slice of the catalog, not an ambiguous combination of a view and a category at the same time. Search matches against both the title and the notes field, and sort has three modes: due date, priority, and recently added. All three live in `taskUtils.js` as small, testable functions rather than inline logic inside the component.

Progress and Stats

The ring in the header is plain SVG — two overlapping circles, where the second one's `stroke-dashoffset` is calculated from the completion percentage. It felt more honest to build it by hand than to add a charting library for one shape.

Deleting Safely

Delete does not happen on the first click. It opens `ConfirmDialog` with the task's title in the message, so you know exactly what you are about to lose. This was one of the additions I made that was not strictly asked for, but I have lost data to a misclick in enough other apps that it felt worth the extra thirty lines of code.

Light and Dark Mode

Theme is detected from the system preference on first load (`prefers-color-scheme`), then remembered in `localStorage` once you toggle it manually. Because of the token system, dark mode



is not a separate set of components — it is the same markup with a different data-theme attribute on the `<html>` element.

Responsiveness and Accessibility

Below 860px the sidebar becomes an off-canvas drawer with a scrim behind it instead of squeezing into the layout. Every interactive element has a visible focus ring (I did not touch the default outline-suppressing habit some CSS resets fall into), the checkbox and priority tag both carry proper aria attributes, and Escape closes both the form and the delete dialog. None of this is exotic, but it is the difference between an app that merely works and one that is comfortable to actually use.

Component Reference

A short map of what each file is responsible for, since a folder full of .jsx files is not self-explanatory on its own:

Component	Responsibility
App.jsx	Wires the whole page together — sidebar, header, stats, task list, and the two modals — and owns the theme (light/dark) since that has to live above everything else.
TaskContext.jsx	The single place where task data and UI state (filters, search, sort, which dialog is open) live. Exposes a state hook and an action hook so components never touch the reducer directly.
Sidebar.jsx	Renders the four view "drawers" and the category list, with live counts pulled from the current task array.
Header.jsx	Search box, sort dropdown, theme toggle, and the "New Task" button. Also shows the current view title.
StatsPanel.jsx	The progress ring plus four counters (open, due today, overdue, completed), computed fresh from the task list on every render.
TaskList.jsx	Applies the active filter and sort to the task array, then decides whether to show cards or one of the two empty states.



TaskCard.jsx	One task, drawn as an index card: the punch-hole edge, the checkbox circle, the priority tag, and — when a task is late — the rotated "Past Due" stamp.
TaskForm.jsx	The add/edit modal. Same form handles both cases; it just checks whether an id was passed in.
ConfirmDialog.jsx	A small "are you sure" step before a task is actually removed, so a misclick does not cost you data.
EmptyState.jsx	Two different empty messages — one for a brand-new catalog, one for a filter that has nothing to show — so the app never displays a blank rectangle.

Decisions I Went Back and Forth On

A few things took more than one attempt to get right, and I think they are worth writing down rather than pretending the first version was the final one.

- Overdue logic: my first pass compared full Date objects, including the current time of day, which meant a task due "today" would flip to overdue the moment the clock passed midnight of the due date at whatever hour I happened to be testing. I switched to comparing date-only values (time zeroed out) so "today" and "overdue" behave the way a person would actually expect.
- Filter and category interaction: originally I let view and category filters apply at the same time, which meant it was possible to select "Completed" and "Work" and get an empty list with no clue why. Now picking one resets the other, and the header title always reflects exactly what you are looking at.
- Card layout on small screens: the hover-to-reveal edit and delete icons work fine with a mouse but are invisible on a touch device that has no hover state. Below 640px those icons are always visible instead of appearing on interaction.
- Splitting task data from UI state in Context: my first version had one big reducer that mixed tasks and UI flags together, which worked but meant every dispatch — even opening the search box — technically touched "the data." Separating them into two contexts made the mental model clearer and avoided unnecessary re-renders.

Running the Project

Everything needed is in the project folder. From inside it:



- `npm install` — installs React, Vite, and the small set of dev dependencies.
- `npm run dev` — starts the local dev server (defaults to `http://localhost:5173`).
- `npm run build` — produces a static, production-ready bundle in `dist/`, which can be deployed to any static host since there is no backend to run alongside it.

The app also passes a clean ESLint run and builds without warnings, which I checked before calling it finished.

What I'd Add With More Time

If this were going further than a Day 25 exercise, the next things on my list would be drag-and-drop reordering within a view, subtasks or checklists inside a card, and swapping `localStorage` for a real backend so the catalog could follow you across devices instead of living in one browser. None of those change the fundamentals this project was meant to practice, which is part of why I left them out for now — I wanted the submission to be a clean demonstration of the concepts rather than a half-finished version of a bigger app.

Closing Note

The brief asked for a task manager built on React fundamentals, and that is exactly what this is underneath the design choices — components, state, props, hooks, forms, and conditional rendering, all doing ordinary, well-understood things. The card catalog idea was the part I enjoyed most, mostly because it gave every design decision a reason to point back to instead of picking colors and spacing arbitrarily. I would rather submit something with a clear point of view than something generic that technically meets the requirements.

THANK YOU!